

Lab 1 Git

The goal of this workshop is to practice configuration management with git.

Git bash

First, you need to install a command-line version of git called git bash on your computer. You can download it from <https://git-scm.com/downloads>. After installation, run git bash.

Configuration

- Run git bash.
- Identify with the commands:
 - `git config --global user.name "SURNAME NAME sXXXXXX"`
 - `git config --global user.sXXXXXX@studenti.polito.it`
- Create a folder for your local repository and move into it.
- Initialize the local repository with `git init`.

Basic concepts of the local repository

- **Difference between working directory, staged area and local repository -**
commands `add`, `status`, `commit`, `log`, `restore`, `diff`.
 - Create a file in the directory (using a text editor) and try to commit. If the staging area is empty, you cannot commit, but git reports the presence of a new 'untracked' file.
 - Add the file to the staging area and check with `git status`.
 - Do commit and then `git status` again.
 - Edit the file with a text editor and check with `git status`.
 - Also try `git diff` to see the lines changed.
 - Do commit (it will not run because the file was not in staging).
 - Add the file and commit.
 - Review commit history with `git log`.
 - Edit a file in two steps: edit it, add it to the staging area (without committing right away), edit it again, and check the status.
 - Remove a file from the working directory (using an operating system command such as `delete` in Windows and equivalent for Linux and macOS).
 - Recover the deleted file with `git restore <file>`.
 - Remove a file with `git rm <file>`, check the status and commit.
 - Attempt to recover the file with `git restore <file>`.

Versioning. Going back and forth between snapshots.

- Retrieve an older version of a file with `git checkout <hashcode>`.
- Go to the most recent version with `git checkout <last hashcode>`.

How commits and snapshots are represented. HEAD.

- Create a new folder, initialize a repository with `git init`, and explore the `.git` and `.git/objects` folders.
- Create a file, add it to the staging area, commit, and then explore `.git` and `.git/objects` again.
- Use `git log` to take note of the 40-letter hash code that identifies the last commit.
- Use `git rev-parse HEAD` and `git rev-parse --short HEAD`.

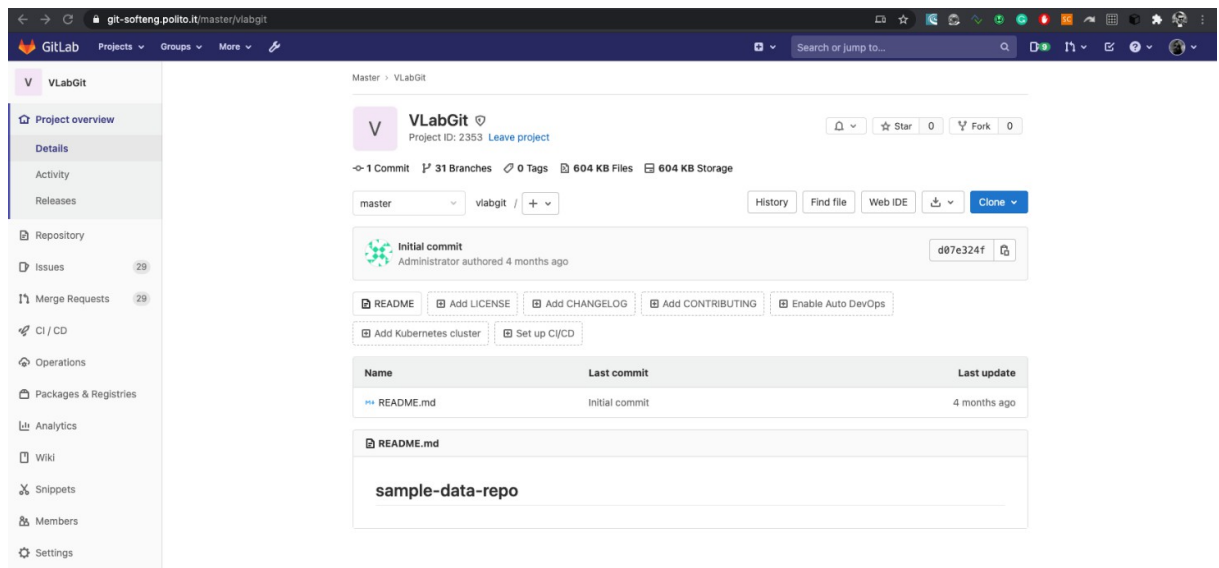
Basic concepts of remote repository

- Set up the account on GitLab
- Configure the ssh key <https://docs.gitlab.com/ee/user/ssh.html>
- Create a new repository: <https://git-softeng.polito.it/projects/new>
- Copy the string [git@git-softeng.polito.it:USERNAME/project.git](https://git-softeng.polito.it/USERNAME/project.git)
 - Where USERNAME is your GitLab username (sXXXXXX), and project is the name of the Project you just created.
- Connect the local repository with the remote repository on GitLab.

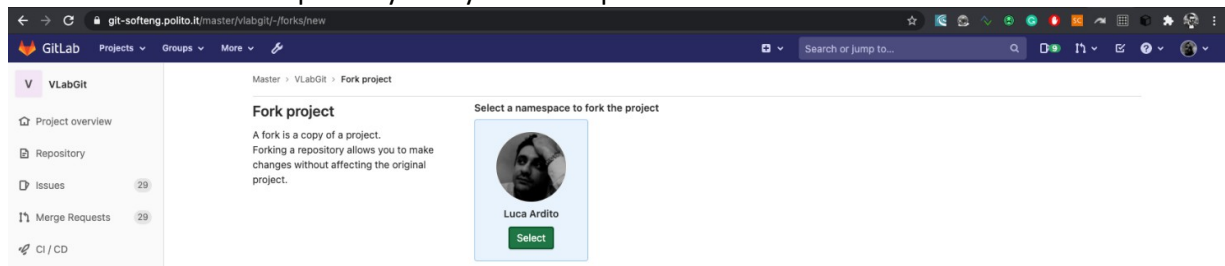
```
mkdir exercise2
cd exercise2
echo "exercise 2">> test.txt
git init
git add test.txt
git commit -m "first commit"
git remote add origin git@git-softeng.polito.it:USERNAME/project.git
git remote -v
git push -u origin main
```

Fork, Clone, Merge request

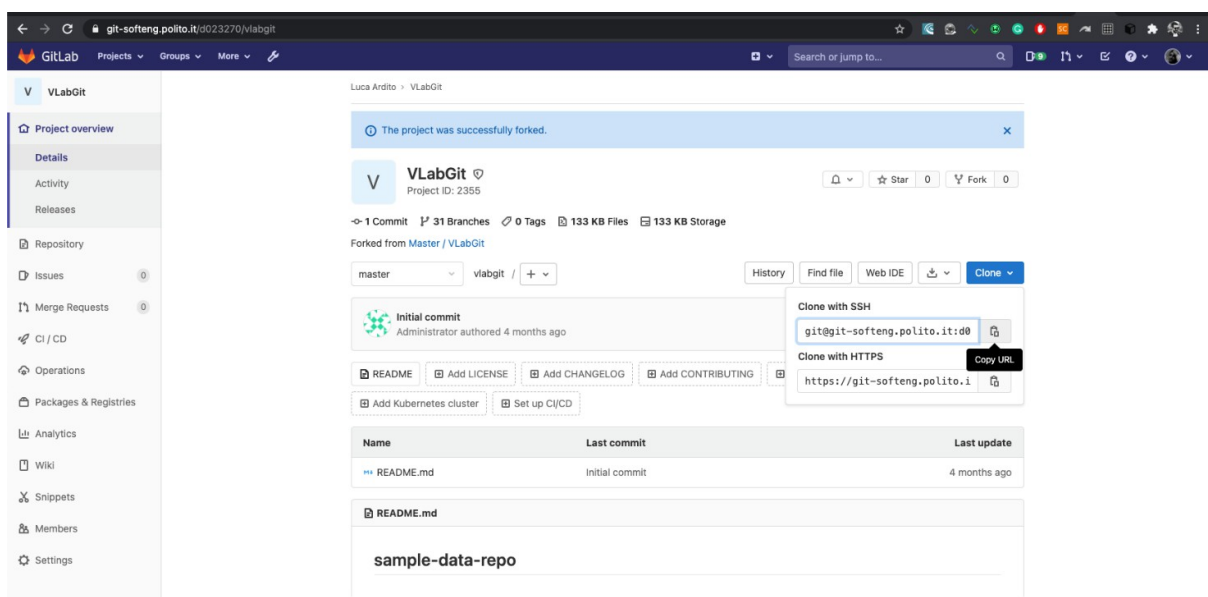
1. Open the link: <https://git-softeng.polito.it/master/vlabgit>
2. Click on Fork



3. Fork the VLabGit repository into your workspace

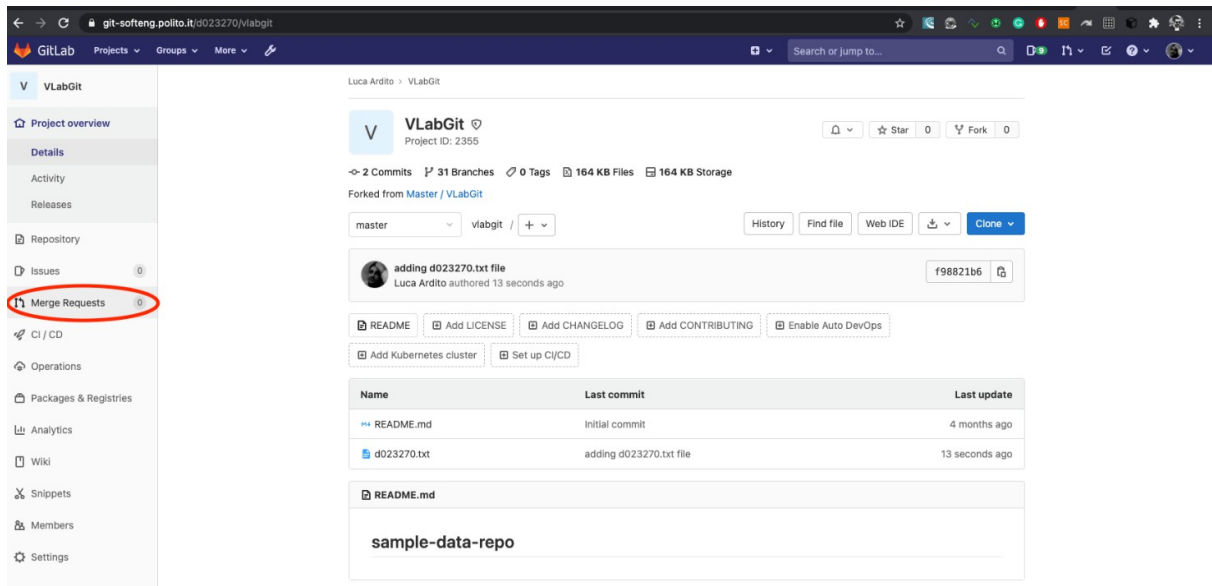


4. Copy the ssh link of your project (the one in your account, not the one in the Master group).

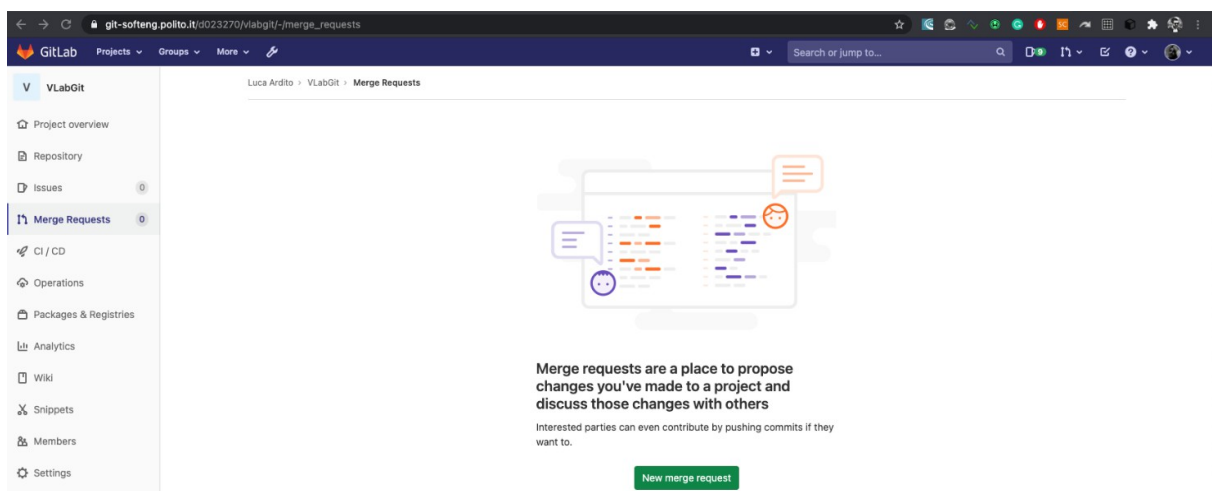


5. Clone the project to your computer with this command `git clone git@git-softeng.polito.it:USERNAME/vlabgit.git`, where USERNAME is your username.

6. Create a new file called sXXXXXX.txt (where XXXXXX is your polity id)
7. Add it to the staging area
8. Commit with the message "adding sXXXXXX.txt file."
9. Push in origin main
10. Merge request



11. Create a merge request



12. Select the branch to merge into the original project and write a comment.

git-softeng.polito.it/d023270/vlabgit/-/merge_requests/new#

GitLab Projects Groups More

Search or jump to...

V VLabGit

- Project overview
- Repository
- Issues 0
- Merge Requests 0
- CI / CD
- Operations
- Packages & Registries
- Analytics
- Wiki
- Snippets
- Members
- Settings

Luca Ardito > VLabGit > Merge Requests > New

New Merge Request

Source branch

d023270/vlabgit master

adding d023270.txt file
Luca Ardito authored 2 minutes ago f98821b6

Compare branches and continue

Target branch

master/vlabgit master

Initial commit
Administrator authored 4 months ago d07e324f

git-softeng.polito.it/d023270/vlabgit/-/merge_requests/new?utf8=✓&merge_request%5Bsource_project_id%5D=2355&merge_request%5Bsource_branch%5D=mas...

GitLab Projects Groups More

Search or jump to...

V VLabGit

- Project overview
- Repository
- Issues 0
- Merge Requests 0
- CI / CD
- Operations
- Packages & Registries
- Analytics
- Wiki
- Snippets
- Members
- Settings

Luca Ardito > VLabGit > Merge Requests > New

New Merge Request

From d023270/vlabgit:master into master/vlabgit:master [Change branches](#)

Title

adding d023270.txt file

[Start the title with Draft:](#) or [WIP:](#) to prevent a merge request that is a work in progress from being merged before it's ready.
Add [description templates](#) to help your contributors communicate effectively!

Description

[Write](#) [Preview](#) **B** *I* `</>`

I have added my id file. Could you please include it in your project?

[Markdown and quick actions](#) are supported [Attach a file](#)

Assignee

Unassigned [Assign to me](#)

Reviewer

Unassigned

Milestone

Milestone

Labels

Labels

Merge options

☐ Squash commits when merge request is accepted. [?](#)

Contribution

☐ Allow commits from members who can merge to the target branch. [About this feature](#)
Not available for protected branches

[Submit merge request](#) [Cancel](#)

Commits 1 Changes 0

26 Jan, 2021 1 commit

adding d023270.txt file
Luca Ardito authored 3 minutes ago f98821b6